

Receipt-Gated Model Serving

Routing to a confidential model host only after a signed admission receipt

Imran Siddique · OPAQUE Systems · October 1, 2026

Technical report, version 1 · Software experiments with a reference implementation

Abstract

A confidential model host has to exist before it can attest, so the Pod comes first. The open question is when clients may reach it. This report describes a mechanism that answers it with evidence. A planner produces a bootstrap Pod that holds only encrypted model material, and produces the Kubernetes Service that routes to it only after it has verified a signed admission receipt for that exact hosting session: pinned signer key, first position in the receipt chain, and equality with six application bindings (tenant, model, encrypted artifact, workload, policy, transport). A signed, terminal lifecycle receipt for the same session removes the route. Because the planner keeps no state, a controller carries a durable, monotonic per-session ledger, so a session that has been revoked can never be routed again from its old admission. We evaluated the planner in an internal repository at two commits, and a reference implementation published with this report. The planner passed its 6 selected tests, rejected six correctly signed receipts that each carried one wrong binding, and, as expected for a stateless component, still accepted an old admission after revocation. The controller, run against a fake Kubernetes API in which every action is a fresh process, kept every revoked session unroutable across all 12 cases of its test matrix. Removing the ledger check, or keeping the ledger in memory, failed 7 of 12 cases. Restoring an old copy of the ledger reopened a revoked session, so the ledger itself needs rollback protection. The evidence is software only; no cluster, confidential GPU or model inference was used.

Scope of this edition. This is an author-prepared technical report, not peer reviewed and not a normative specification. It is the first public description of the mechanism. The evaluated planner lives in an internal OPAQUE repository; its commit identifiers, test counts and file hashes are recorded, its failing tests appear under generic labels, and its source is not published. Every experiment also runs against the reference implementation in the accompanying experiments folder, which needs Python 3.12 and the cryptography package and reproduces every result in section 6. The experiments were rerun on October 1, 2026 in a fresh environment; the record is in experiments/results/rerun-2026-10-01. One change relative to the September 27, 2026 internal draft: this edition found that a revocation issued after the host has served requests did not chain directly to the admission, so the earlier controller refused it and left the Service in place. Section 4.5 describes the fix and section 6.5 the case that tests it.

1. A running Pod is not an admitted host

Kubernetes routes traffic to a Pod through a Service. By default a Service sends traffic to any Pod that matches its selector and reports ready. For an ordinary web server that is fine. For a confidential model host it is too early. The host Pod has to be scheduled and started before it can produce hardware evidence, before a key broker releases the model key, and before it has loaded anything. If the Service already exists at that point, clients can reach a process that has not been admitted, and two systems now disagree: the admission system says "not yet" and the routing system says "ready".

The obvious fix is to delay the Service until something says the host is good. The question is what that something is. A readiness probe is the host reporting on itself. An annotation on the Pod is a mutable string. This design uses the host's own signed admission receipt, checked against values the deployment pinned before the host started.

The same problem runs the other way. When the host is revoked (policy change, attestation key compromise, emergency stop), the route must go and must not come back. A signed revocation record is only useful if whatever controls routing applies it to the right session and remembers it.

The contribution has three parts:

1. A bootstrap Pod specification that carries ciphertext and no key material, and is never restarted in place.
2. A Service gate: the routing object is produced only from an admission receipt that verifies against a pinned key, sits at the start of its chain, names this session, and matches every application binding.
3. A controller rule: a durable, monotonic per-session ledger, so that a revoked session is never routed again, whatever order events arrive in and however often the controller restarts.

2. Related work

Attestation-gated release of secrets is established practice. In Confidential Containers, the Trustee Key Broker Service releases keys and other resources to a guest only after its attestation evidence satisfies policy. [Confidential Containers: Attestation with Trustee](#) Contrast verifies Pod attestation against a manifest and issues service mesh certificates only to workloads whose evidence matches, which gates mesh participation as well as secrets. [Contrast: Attestation](#) Both decide who gets a key or a certificate. The mechanism here decides whether a cluster routing object may exist, and ties that object to one signed record.

For inference services, Azure AI Confidential Inferencing has a key management service release Oblivious HTTP private keys only to GPU virtual machines that present acceptable attestation, so a client's encrypted prompt can only be opened inside an attested host. [Azure AI Confidential Inferencing: Technical Deep-Dive](#) [attested-ohhttp-server](#) Apple's Private Cloud Compute has the client wrap its request key only to nodes whose attested measurements match a release in a public transparency log. [Private Cloud Compute, June 10, 2024](#) Those designs move the gate to the client and to key release, which is stronger than routing alone because a misrouted request still cannot be decrypted. Receipt-gated routing complements them: it keeps the platform from presenting an unadmitted host as a service at all, and it leaves a routing object that names the evidence behind it. A June 2025 design report by Pattern Labs (now Irregular) separates confidentiality of inference data from custody of model weights, which is the asset this design protects. [Confidential Inference Systems: Design principles and security risks](#)

Kubernetes already lets external signals hold a Pod out of Service endpoints. Pod readiness gates (KEP 580) add custom conditions that must be true before a Pod counts as ready. [Kubernetes Pod lifecycle KEP 580: Pod Ready++](#) A readiness gate could carry the same admission decision. The condition, though, is a status field any sufficiently privileged writer can set, and it records no evidence. Here the Service carries the hash of the receipt that authorized it, and the decision is recomputed from the receipt on every repair. Admission-time image verification, such as the Sigstore policy controller, checks signatures and attestations on images before a Pod is admitted. [Sigstore policy-controller](#) That covers what is launched. It says nothing about whether a launched instance later obtained its key.

In RATS terms, the model host is the attester and its signing key is bound into the hosting grant, the planner and controller are relying parties, and the admission receipt plays the role of an attestation result in the passport model. [RFC 9334, Remote ATtestation procedureS \(RATS\) Architecture](#) The word "receipt" here means a signed record emitted by the host. It is not a SCITT transparency receipt.

3. System and threat model

The parts are a hosting grant, a model host, a manifest planner and a controller that applies the planner's output to the cluster API.

The **hosting grant** is what the deployment approved. It holds eight SHA-256 digests: a session nonce, the tenant fingerprint, the model weights, the encrypted model artifact, the workload, the policy, the transport, and the receipt signing key. The **session identifier** is the SHA-256 of the grant's canonical JSON, so any change to any of the eight values produces a different session.

The **model host** runs inside the confidential environment. On start it compares measured bindings with the grant, obtains the model key through attested release, and signs its first receipt. The host refuses to start twice. Recovery after a crash means a new session nonce, a new grant, a new signing key and a new receipt chain.

The **planner** takes the grant, a Kubernetes host configuration and an independently pinned receipt public key. It produces manifests and never calls the cluster API.

The **controller** applies manifests, keeps the lifecycle ledger and reconciles drift.

The adversary may alter a receipt, replay a valid receipt from another session or with other bindings, replay an old admission after revocation, deliver events out of order or twice, crash the controller at any point, make a cluster write fail, swap an image tag, and edit or create Services out of band. Trusted: the receipt signing key inside the host, the grant and pinned key as configured, the controller's process and ledger storage, and the cluster's own enforcement of Service selectors. An administrator who can write Services directly can still create a route; the controller's reconciliation removes it on its next pass but cannot prevent it.

4. Mechanism

4.1 Receipts

Each receipt is a JSON object with these fields: kind, session_id, sequence (starting at 1), previous_receipt_hash (null for the first receipt), issued_at (UTC, ISO 8601), event (admission, inference or lifecycle), state (locked, serving, stopped, expired, revoked or failed), decision (allow or deny), reason, and the six application bindings tenant_fingerprint, weights_hash, encrypted_artifact_hash, workload_hash, policy_hash and transport_hash. Inference receipts add a request identifier, a prompt digest and a result digest, never the prompt or result.

The signature is Ed25519 over the canonical JSON of every field except the verification block: keys sorted, no insignificant whitespace, UTF-8. The verification block carries the algorithm, the base64url public key and the base64url signature. The **receipt hash** is the SHA-256 of the canonical JSON of the complete receipt, including its verification block, and the next receipt's previous_receipt_hash is that value. Every served request therefore moves the head of the chain.

A successful admission receipt has event admission, sequence 1, no predecessor, state serving, decision allow and reason admitted. A failed start still emits a signed admission receipt, with state stopped, decision deny and a reason such as binding-mismatch:workload_hash. Revocation unloads the runtime, wipes key custody and emits a lifecycle receipt with state revoked, decision deny and one of three reasons: policy-revoked, attestation-key-revoked or emergency-revocation.

4.2 The bootstrap Pod

The Pod specification requires an image pinned by `@sha256:` digest, mounts the encrypted model artifact read-only from a `PersistentVolumeClaim`, and contains no `Secret` volume and no key material. It sets `restartPolicy: Never`, so a crashed host is not silently restarted under the same identity. It also sets `automountServiceAccountToken: false`, disables host network, PID and IPC namespaces, runs as non-root with the `RuntimeDefault` seccomp profile, drops all capabilities, forbids privilege escalation and uses a read-only root filesystem. It requests a confidential runtime class and one GPU. The Pod is labelled with the application name and a session label, and annotated with the full session identifier, workload hash and encrypted-artifact hash.

These are construction choices a reviewer can read in the manifest. They do not replace cluster admission policy.

4.3 The Service gate

The planner returns a `Service` only when the admission receipt passes all of these checks, in order:

1. The receipt is well formed, every digest field is a lowercase SHA-256 digest, and no unknown fields are present.
2. The signing key in the receipt equals the independently pinned key, and the signature verifies.
3. `session_id` equals the grant's session identifier.
4. `previous_receipt_hash` is null, so this is the first receipt in the chain.
5. `event`, `sequence`, `state`, `decision` and `reason` are `admission`, `1`, `serving`, `allow` and `admitted`.
6. Each of the six bindings equals the grant's value (constant-time comparison for strings).

Any failure raises an error that lists every mismatched field, and no `Service` is produced. The `Service` is of type `ClusterIP`. Its selector is the application name plus a session label, and it is annotated with the admission receipt hash and the full session identifier. A reviewer looking at the live `Service` can find the exact receipt that authorized it.

Step 6 exists because a valid signature is not enough. A correctly signed receipt can still describe a different model, a different policy or a different transport, and the planner must compare it with what was approved rather than with what the receipt claims.

4.4 The revocation target

Given a revocation receipt and the expected predecessor hash, the planner verifies signature, pinned key, session and chain position, then requires event lifecycle, state revoked, decision deny and one of the three revocation reasons. It returns the `Service` name, namespace and the revocation receipt hash as a delete target. It does not call the API and cannot confirm removal.

4.5 The controller and its ledger

The planner is stateless, and section 6.4 shows why that matters: it accepts an old admission receipt after revocation, because nothing in that receipt changed. The controller adds the missing memory. Its ledger has one row per session: `session_id` (primary key), `state` (admitted or revoked), the admission receipt hash, the revocation receipt hash and the admission receipt bytes. The prototype uses SQLite.

Admission. Parse the receipt once from bytes and use only that parsed copy afterwards. Verify it through the unchanged planner. In one write transaction, refuse if the session's row says revoked; otherwise insert the row as

admitted, keeping any existing row. Then create the Pod, then the Service. An existing Service that differs from the expected one in spec or annotations is replaced.

Revocation. Verify the record against the pinned key and the session, and require a lifecycle event with decision deny and a terminal state (revoked, failed, stopped or expired). Write the revoked tombstone first, then delete the Service. A tombstone is never cleared. The controller does not require the revocation to chain directly to the admission: served requests move the chain head (section 4.1), and deleting a route can only reduce access, so any authentic terminal record for the session is sufficient. Chain continuity remains available for audit. The September 27 draft's controller required the direct link and, as section 6.6 shows, then refused a real revocation after a single served request.

Reconciliation. For each known session: if the ledger says revoked and a Service exists, delete it. If the ledger says admitted, rebuild the expected Service from the recorded admission bytes, re-verifying through the planner each time, and replace a missing or drifted Service. Reconciliation never reads its inputs from live cluster objects.

A revoked session can only become routable again if its tombstone disappears. That is why section 6.6 restores an old copy of the ledger.

5. Why each check is needed

Signature verification stops modification by anyone without the host's key. The pinned key stops a receipt signed by a different host from counting. The session check stops a valid admission from another session (another nonce, another signer) from opening this one. The first-position check stops a later receipt in the chain, such as a renewal, from being presented as the admission. The binding checks stop a correctly signed receipt that describes the wrong model or policy, which section 6.3 tests directly.

None of those checks can tell an old admission from a current one, because the old admission is still authentic. Only state outside the receipt can. The ledger is that state, and it is monotonic: admitted can become revoked, and revoked never becomes anything else. With the tombstone written before the delete, a crash between the two leaves a revoked session with a stale Service that the next reconciliation removes, and never a Service with no tombstone.

6. Evaluation

6.1 Setup and reproduction

The evaluated planner is in an internal OPAQUE repository, at commit 52632cce4d21098cf00283075254c3e0de2a461a (August 21, 2026) and at its main branch on October 1, 2026, commit 3d5b48a5294bc73d42a2bd4fbd23415027e773a7. The planner and model host files are byte-identical at both commits (SHA-256 prefixes 72457274a0a3 and e36770589e0c). Each run exports the commit with git archive into a temporary directory and runs it unmodified.

The reference implementation in experiments/reference/receipt_gate.py restates the grant, host receipt chain, verifier and planner in about 340 lines. It differs from the evaluated planner in four ways: it uses a neutral annotation prefix, it emits inference receipts without running a model, it has its own 8 unit tests, and its planner refuses a pinned key whose SHA-256 does not match the grant's receipt_signing_key_hash. The evaluated planner does not perform that cross-check (section 7).

The experiment scripts run unchanged against either implementation through a small backend module. python rerun.py --output <new dir> runs the reference target. With --evaluated-repo and --commit, it also runs the evaluated planner at each commit and records only counts and hashes for that target; in the published record its

failing tests carry generic labels. The October 1, 2026 run used Windows 11, Python 3.12.10, pytest 9.1.1, pytest-asyncio 1.4.0 and cryptography 50.0.1 in a new virtual environment. No cluster, cloud resource, network service or confidential hardware was used.

6.2 Planner tests and control removals

Target	Selected tests	Admission check removed	Restart policy set to Always
Evaluated, 52632cc	6 passed, 97 deselected	1 failed	2 failed
Evaluated, 3d5b48a (main)	6 passed, 109 deselected	1 failed	2 failed
Reference implementation	8 passed	2 failed	1 failed

The evaluated tests check the bootstrap manifest (no key, no Secret, restart Never, no service-account token, read-only root, capabilities dropped), Service creation for a pinned admission with the receipt hash annotation, refusal of a tampered and of a denied admission, rejection of an image tag without a digest, the chained revocation delete target with a wrong-predecessor refusal, and a documented orchestration self-test. The extra deselected tests at main are unrelated suites added after the pin. The failing tests under each removal are the same as in the September 23 and September 27 internal runs. These removals show the tests depend on those two rules. They are not a mutation-coverage measure.

6.3 Correctly signed, wrong bindings

Starting from a valid admission receipt, the experiment changed one binding at a time (tenant, model weights, encrypted artifact, workload, policy, transport) and re-signed each altered receipt with the host's real key. Each of the six verified as a signature. The planner refused all six, and each refusal named the changed field. The result was identical at both commits and on the reference implementation.

6.4 Stale admission after revocation

The same experiment then revoked the host and submitted the original admission receipt to the unchanged planner. It returned a Service. This is the expected behaviour of a stateless planner and the reason the controller exists. Identical on all three targets.

6.5 Controller against a fake Kubernetes API

The fake API is a separate process serving namespaced Pods and Services with create, get, replace, delete and list, 409 on duplicate create, resource versions, and injectable create failures. Every controller action runs as a new process, so every step is also a controller restart. Seven hosting sessions each use their own generated signing key. Hypothesis: with an intact ledger, no revoked session has a Service at any check. Criterion: all 12 cases.

Case	Observation
Admission	Pod and Service created; Service carries the admission receipt hash
Duplicate admission	Service unchanged, same resource version
Revocation	Service deleted; ledger tombstoned
Restart, then replay of the old admission	refused; no Service
Revocation processed before admission	admission refused; never routable

Case	Observation
Service selector changed out of band	reconcile restores the expected selector
Service for a revoked session created out of band	reconcile deletes it
Service create fails after the Pod exists	no Service; reconcile completes it from the ledger
Create fails, then revocation arrives	reconcile and later replay never create a Service
Caller's receipt object replaced after verification	applied Service still names the verified receipt
Revocation after a served request (new)	Service deleted although the revocation does not chain to the admission
Final sweep after another restart	all four revoked sessions have no Service

All 12 cases met the criterion on all three targets. The first 11 cases are the September 27 matrix and gave the same results as that run.

6.6 Controls and the rollback limit

Control	Cases passed	Failing cases
Ledger check removed (planner only, no reconciliation)	5 of 12	old admission recreates the Service after revocation, plus every reconcile case
Ledger kept in memory, lost at each restart	5 of 12	the same 7 cases
Revocation must chain directly to the admission (September 27 rule)	10 of 12	revocation after a served request, and the final sweep

Every control failed its criterion, on all three targets. The third control is the gap this edition found: one served request moved the chain head, the September 27 controller refused the genuine revocation with "receipt chain predecessor mismatch", and the Service stayed routable.

After the durable run, the experiment restored a copy of the ledger taken before any revocation and replayed the old admission. It created a Service again. The ledger moves the trust assumption: it has to be protected against rollback, for example by a monotonic counter in hardware or an append-only store the controller's operator cannot rewind.

Two further observations. The bootstrap Pods of revoked sessions that had been admitted stayed in the API after their Services were deleted, because this controller removes routing and leaves workloads alone; the host itself has already unloaded and wiped custody on revocation. And the receipt-replacement case mainly shows that the controller parses the receipt once from bytes. It is a design check, not a race that was found and fixed.

7. Limits

The evidence is software. No cluster, Service proxy, network policy, confidential GPU or model inference was involved, and no revocation latency was measured. Deleting a Service also does not close connections that are already open or other paths to the Pod; an isolated-cluster run should check direct Pod reachability and the host's own refusal state separately from routing. It was not run because no kind, k3d or Docker installation was available on the run machine.

The Service selector uses the first 16 hex characters (64 bits) of the session digest as a label, and Kubernetes labels can be set by anyone with Pod write access. The full session identifier is only an annotation. Selector integrity

depends on cluster write controls, not on the receipt.

The evaluated planner trusts the pinned key it is given and does not check it against the grant's `receipt_signing_key_hash`. The reference implementation adds that check at construction; a deployment of the evaluated code should do the same.

Canonical JSON here is sorted-key compact JSON as produced by Python's standard library, not RFC 8785. An implementation in another language must reproduce the same bytes.

8. What a team can check

A deployment that adopts this design can verify it without trusting the controller's logs. For each live Service, read the admission receipt hash annotation, fetch that receipt, and verify it against the pinned key and the grant: it must be the first receipt of the session named in the session annotation, and every binding must match. For each session that has emitted a terminal lifecycle receipt, there must be no Service. Then restore a backup of the ledger taken before a revocation and replay the old admission. The prototype in this report fails that test (section 6.6), so whatever stores the ledger has to pass it.

The experiments folder has the reference implementation, the backend module, the boundary and controller experiments, the rerun script and the October 1, 2026 record, with SHA-256 sums for each output file.